

## Part C · 前端可视化 (5 页)

**定位：**讲前端架构、技术栈选型、状态与数据通信、可视化体系、三维地理渲染、路由工程化与交互规范。不含实操代码。

### 页面总览

| 页码   | 标题                  | 一句话                                            |
|------|---------------------|------------------------------------------------|
| C-P1 | 前端架构与技术栈            | React 19 + TS 5.9 + Vite 8 + AntD 6 + Tailwind |
| C-P2 | 状态管理与数据通信           | Zustand + Axios + 双通道（REST + 实时）               |
| C-P3 | 可视化体系（ECharts）      | 图表矩阵 + 增量渲染 + LTTB 降采样                         |
| C-P4 | 地理信息与三维渲染（Three.js） | 四图层叠加 + Mercator + 相机联动                        |
| C-P5 | 路由权限、工程化与交互规范       | Lazy + manualChunks + 四级预警色                    |

### C-P1 · 前端架构与技术栈

C-P1

前端架构与技术栈

框架层

构建层

UI 层

可视化层

目录分层 (src/)

React 19

· 并发特性

· Suspense

· 函数组件

TypeScript

5.9

Vite 8

· ESM 冷启

· HMR

· 按需编译

· manualChk

Ant Design 6

· 企业组件

+

Tailwind CSS

· 原子样式

· 设计 Token

ECharts 6 (2D 图表矩阵)

· 折线 · 散点 · 热力 · 桑基

· 雷达 · 仪表盘

Three.js (3D 地理场景)

· 四图层叠加

· Mercator 投影

pages/

12 个业务页面模块

components/

复用组件库 (11 个子目录)

services/

API 服务层 (api.ts + mqtt)

store/

Zustand 状态切片

hooks/

自定义 Hook

types/

TypeScript 类型声明

assets/

静态资源 (字体 / 图标 / HDR / GLB)

1 / 7

设计 Token  
主色 #0891b2    辅色 #10b981  
警示 #f59e0b    危险 #ef4444  
暗色模式 / 大屏模式 / 管理模式三种视觉基调

讲稿要点

- **框架选型**：React 19 + TypeScript 5.9 + Vite 8——React 19 的并发特性配合 Suspense 让大屏组件渐进渲染不卡顿；TS 5.9 严格类型系统约束跨页面数据流；Vite 8 的 ESM + 按需编译把冷启动时间压到秒级
- **UI 组件体系**：**Ant Design 6 + Tailwind CSS 混用**——AntD 提供企业级复杂组件（表格、表单、抽屉、弹窗），Tailwind 提供原子级布局与间距样式；两者通过设计 Token 对齐色板
- **目录分层**：`pages/` 12 个业务模块（大屏 / 站点 / 告警 / 溯源 / AI 对话 / 报告 / 管理...）、`components/` 11 个复用组件子库、`services/` API 层、`store/` Zustand 切片、`hooks/`、`types/`、`assets/`
- **设计 Token**：主色 #0891b2（水蓝）、辅色 #10b981（生态绿）、警示 #f59e0b（橙）、危险 #ef4444（红）；对应四级预警色语义；暗色模式贯穿大屏与管理台
- **响应式与模式切换**：大屏模式（高对比、全屏可视化）与管理模式（紧凑表格、精细交互）通过布局容器分流
- **技术事实**：严格只使用函数组件 + Hooks 范式；所有跨组件共享状态均走 Zustand store，不混用 Redux 或 Context

下一页过渡：框架选型完，数据怎么从后端到组件、怎么跨页面同步？进入状态管理与通信。

C-P2 · 状态管理与数据通信

C-P2

状态管理与数据通信

Zustand Store 切片设计

|                 |                       |
|-----------------|-----------------------|
| useStationStore | 站点列表 · 选中站点 · 筛选条件    |
| useMetricStore  | 实时水质缓存 · 时间范围 · 降采样选项 |
| useAlertStore   | 告警计数 · 四级分布 · 实时推送流   |
| useAuthStore    | 用户会话 · 角色权限 · Token   |
| useMapStore     | 地图视角 · 图层开关 · 联动状态    |

为什么选 Zustand (vs Redux / Context)

- 样板代码少 · Hooks 化 API · 无 Provider 包裹
- 基于 selector 精准订阅 · shallow 浅比较减重绘
- 异步更新无需 thunk / saga

数据通信：双通道协作

REST (历史)

Axios 实例 (baseUrl · 拦截器 · 超时 30s)



讲稿要点

- **Zustand 切片设计**: 按业务域拆分 5 个 store——站点、实时水质、告警、用户会话、地图; 每个切片只暴露必要的 getter/setter, 组件通过 selector 精准订阅某几个字段
- **选型理由** (对比 Redux) : Zustand 样板代码少、无需 Provider、基于 selector 做**精准订阅 + shallow 浅比较**, 天然适合大屏场景下「同一数据多组件展示」的性能需求
- **Axios 封装**: 统一 **baseURL**、30 秒超时、请求拦截器注入 **Authorization: Bearer <token>**、响应拦截器处理 401 自动登出 + 统一错误 toast; 业务代码只关心 **api.get(...)** / **api.post(...)**
- **AbortController**: 页面切换时取消上一个页面未完成的请求, 避免重复请求竞态污染当前页面 store
- **双通道协作** (REST + WebSocket) :
  - REST 负责拉取**历史数据快照** (图表初始加载、站点详情打开)
  - WebSocket 负责推送**增量事件** (新告警、新水质点、异常检出)
  - 前端订阅后直接写 Zustand 切片, 大屏 + 详情面板共享同一份数据源
- **MQTT 接入口径** (补充说明) : 前端**不直接连 EMQX 的 MQTT over WebSocket**, 而是通过后端 REST **/api/v1/mqtt/\*** 管理「外部 MQTT 数据源」连接配置; 真正的实时推送走后端 → 前端 WebSocket, 由 Nginx 通用 **/ws/** 透传

下一页过渡: 数据到了 store, 下一步看怎么画出来。

C-P3 · 可视化体系 (ECharts)



|       |                            |
|-------|----------------------------|
| 多指标对比 | 雷达 (radar · 标准化坐标)         |
| 时间×指标 | 热力 (heatmap · 时刻 × 指标矩阵)   |
| 流域物质流 | 桑基 (sankey · 节点=站点 · 边=通量) |
| 综合评分  | 仪表盘 / 极坐标 (gauge · radial) |
| 污染物分布 | 柱状堆叠 / 饼图                  |

#### 实时增量渲染

```
chart.setOption(newOpt, { replaceMerge: ['series'] })
```

- 仅替换 series 数组 · 保留 axis / tooltip / grid 配置
- 避免全量 redraw · 帧率稳定 @30fps+

#### 大数据量降采样: LTTB 算法

Largest-Triangle-Three-Buckets

- 将 N 点分成 K 个桶
- 每桶取让三角面积最大的那个点
- 保留原始形态特征 (峰谷不丢)
- $N=10w \rightarrow K=500$ , 可视无损

#### 图表联动



### 讲稿要点

- **图表矩阵**: 按「场景 → 图表」维度组织 ECharts 6 的使用
  - 单站趋势 → 多指标 overlay 折线 + markLine 国标阈值
  - 异常标记 → 散点叠加折线
  - 多指标对比 → 雷达图标准化坐标
  - 时间 × 指标 → 热力图矩阵
  - 流域物质流 → 桑基图 (节点是站点、边是污染物通量)
  - 综合评分 → 仪表盘 / 极坐标
- **实时增量渲染**: 大屏场景下每秒都有新数据点, 用 `setOption(opt, { replaceMerge: ['series'] })` 只替换 series 数组、保留 axis / tooltip / grid 配置, 避免全量重绘导致的掉帧
- **LTTB 降采样** (Largest-Triangle-Three-Buckets): 当单站历史查询返回 10 万点时, 画布只有 1000 像素宽, **盲目全画没意义**; LTTB 将 N 点分成 K 个桶, 每桶保留让相邻三点三角面积最大的那个点——既压到画布分辨率级别, 又保留峰谷形态
- **图表联动**: 共享 `axisPointer` + 同步 `tooltip`——折线、散点、热力三个图表在同一时间轴上, 鼠标 hover 任一图表, 另两个同步定位
- **技术事实**: 所有图表组件都封装成 React 组件, 通过 `useEffect` 监听 store 切片变化触发 `setOption`; 图表实例用 `useRef` 持久化, 避免组件重渲染销毁

下一页过渡：2D 图表讲完，水环境场景还有一条重要的可视化线——地理信息。

C-P4 · 地理信息与三维渲染 (Three.js)

C-P4

地理信息与三维渲染 (Three.js)

场景组织：四图层叠加

Layer 4 站点图层 (Sprite / Mesh 实例 · 四级预警色)

▲

Layer 3 路网矢量 (Line · 等级分色)

▲

Layer 2 河网矢量 (Line · 蓝色粗细按流量)

▲

Layer 1 底图平面 + 法线贴图 (高程细节)

地理对齐：  
经纬度 (lon, lat) —Mercator—> (x, y)  
                                  ↓  
                                  Three.js 世界坐标 (单位: 米)

Z-Fighting 规避：相对尺度 + 层级偏移  
河网高度 +0.01  路网高度 +0.02  站点高度 +0.05

相机控制 (OrbitControls 定制)

- 默认视角：赣南地区直立俯视
- 使用 Y 轴作为垂直方向 (非 Z 轴)
- 最小仰角 10°防止翻转 · 最大仰角 89°防止从下看
- 缩放限制：minDistance / maxDistance

站点交互：  
raycaster 命中站点 Sprite  
    ↓  
相机 tween 平滑过渡到站点位置 (0.8s easeInOutCubic)  
    ↓  
右侧面板切换到对应站点详情 (Zustand store 联动)

数据源：  
scripts/extract-rivers.mjs  离线从 OSM 提取河网矢量  
scripts/extract-roads.mjs  离线从 OSM 提取路网矢量  
public/hdr/  HDR 环境贴图  
public/model/glb/  GLB 三维模型

- **四图层叠加**：底图平面 + 法线贴图（还原高程细节） → 河网矢量（粗细映射流量） → 路网矢量（等级分色） → 站点图层（四级预警色实时渲染）
- **地理对齐**：经纬度先经 **Mercator 投影** 映射到平面坐标，再按项目约定的单位尺度（米）放进 Three.js 世界坐标；项目地理定位在**赣南地区**，默认视角直立俯视
- **关键坐标约定**：Three.js 中 Y 轴作为垂直方向（不是 Z 轴），相机设置与光照方向全部按此约定；历史上有过「使用 Z 轴导致视角翻转」的坑，专门记在约束里
- **Z-Fighting 规避**：四个图层用相对尺度 + 微小高度偏移（河 +0.01、路 +0.02、站 +0.05）错开，避免多边形 Z 同值的深度闪烁
- **相机控制**：基于 OrbitControls 做定制——最小仰角限制防翻转、最大缩放距离防迷失；默认直立俯视符合调度员对流域全貌的心智模型
- **站点交互**：用 Three.js **Raycaster** 命中站点 Sprite；点击后相机 tween（0.8s easeInOutCubic）平滑过渡到站点位置，右侧面板通过 Zustand **useMapStore** + **useStationStore** 同步切换详情
- **数据来源**：用 **scripts/extract-rivers.mjs** 和 **scripts/extract-roads.mjs** 离线从 **OSM 提取** 河网与路网矢量，避免运行时依赖在线瓦片；HDR / GLB 资产分别放在 **public/hdr/** 与 **public/model/glb/**

**下一页过渡**：可视化层讲完，最后看工程化与交互规范怎么串起整个前端。

## C-P5 · 路由权限、工程化与交互规范

| C-P5                                                                                                                                                                                                                                                                                                    | 路由权限、工程化与交互规范 |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| React Router 7 嵌套路由                                                                                                                                                                                                                                                                                     |               |
| <div>&lt;Layout&gt;</div> <div><div><div>/dashboard</div><div>/stations/:id</div><div>/alerts</div><div>/tracing</div><div>/ai</div><div>/reports</div><div>/admin/*</div></div><div>大屏</div><div>站点详情</div><div>告警中心</div><div>溯源分析</div><div>AI 对话</div><div>报告中心</div><div>管理后台（角色隔离）</div></div>    |               |
| 代码分割：React.lazy + Suspense                                                                                                                                                                                                                                                                              |               |
| 页面级懒加载 · 按路由切分 chunk · 首屏只加载 Layout + 当前页                                                                                                                                                                                                                                                               |               |
| 路由守卫：                                                                                                                                                                                                                                                                                                   |               |
| 登录态（token 存在） → 角色权限（useAuthStore.role） → 进入路由                                                                                                                                                                                                                                                          |               |
| Vite 构建优化                                                                                                                                                                                                                                                                                               |               |
| <div>• 代理：/api /health → http://127.0.0.1:8000（强制 IPv4）</div> <div>• manualChunks：</div> <div><div>vendor</div><div>echarts</div><div>three</div><div>• react / react-dom / react-router</div><div>• echarts 全量单独拆</div><div>• three.js + OrbitControls 单独拆</div></div> <div>• 产物压缩：gzip + brotli</div> |               |
| 首屏优化：                                                                                                                                                                                                                                                                                                   |               |

关键 CSS 内联 · 字体子集化 (FontMin)  
 图片 WebP · HTTP/2 资源预加载 (preload)

#### 四级预警色语义 (跨 Part 统一)

|          |         |            |
|----------|---------|------------|
| critical | #ef4444 | 严重 · 需立即响应 |
| high     | #f59e0b | 高危 · 需快速处置 |
| medium   | #facc15 | 中等 · 需关注   |
| low      | #10b981 | 低危 · 记录观察  |

#### 可访问性 / 交互规范

- ARIA role 标注: alert / status / log
- 键盘可达: Tab 顺序 · Esc 关闭弹层
- 过渡时长: 300ms 常规 · 800ms 大场景切换
- 缓动曲线: easeInOutCubic 主视觉 · linear 实时数据流
- AI 对话入口: 全局右下 FAB · 异常检出后自动弹出询问

### 讲稿要点

- **React Router 7 嵌套路由**: 统一 `<Layout>` 容器 + 各子页面; 路由表按业务模块分组, 管理后台 `/admin/*` 单独角色隔离
- **代码分割**: 每个页面组件用 `React.lazy()` 包裹 + `<Suspense fallback={...}>` 兜底 loading; 按路由切分 chunk 后首屏只加载 Layout + 当前页面
- **路由守卫**: 两级校验——先看登录态 (Token 是否存在), 再看角色权限 (Zustand `useAuthStore.role` 与路由元数据匹配); 未通过即重定向至登录页或 403
- **Vite 构建优化**:
  - **代理**: `/api`、`/health` → `http://127.0.0.1:8000`——**强制 IPv4** 规避 Windows 下 localhost 解析为 IPv6 `:::1` 的 socket hang up 问题 (跨平台排障经验)
  - **manualChunks**: 把 `vendor` (React 栈)、`echarts`、`three` 三个大块拆成独立 chunk, 首次访问后可被浏览器缓存命中
  - 产物用 gzip + brotli 双重压缩
- **首屏优化**: 关键 CSS 内联进 HTML、字体做子集化 (只保留中文常用字 + 英数字)、图片统一 WebP、HTTP/2 `preload` 关键资源
- **四级预警色跨栈统一**: `critical #ef4444 / high #f59e0b / medium #facc15 / low #10b981`——与 Part B 的告警分级、Part D 的图计算输出严格一致, 避免跨页面色彩语义冲突
- **交互规范**: ARIA role 规范化 (alert / status / log 分工明确); 过渡时长分档 (常规 300ms、大场景 800ms); 缓动曲线约定 (主视觉 easeInOutCubic、实时数据流 linear 避免抖动)
- **AI 对话入口**: 全局右下角 FAB (悬浮按钮) 固定位置; 当系统检出新异常时自动弹出询问「是否分析?」, 把异常上下文直接带入对话

**衔接到 Part D**: 「到这里, 前端已经把数据以图表、地图、异常标记、告警清单的形式呈现给值班员。但展示只是可见, 不是可理解。下一部分看 AI 怎么把这些数据变成决策。」